

X.509 Authentication Service Module 1

Prepared by

Jesna J S

Assistant Professor, CSE TKMCE

X.509 Authentication Service

- The **X.509 authentication service** is a security mechanism that **uses X.509 digital certificates to authenticate the identity of users, devices, or servers in a network.**
- X.509 is a **standard defined by ITU-T** (International Telecommunication Union – Telecommunication Standardization Sector) for **public key certificates.**

An X.509 certificate binds:

- ✓ **An identity** (user, server, organization)
- ✓ **To a public key**
- ✓ Using a **trusted Certificate Authority (CA)**

- X.509 is an ITU-T recommendation and **part of the X.500 series**, which defines a directory service.
- The X.500 directory is a server or a distributed set of servers that maintains a database of user information.
- Information stored includes:
 - ✓ Mapping of user name to network address
 - ✓ Other attributes and user-related information
- X.509 defines a framework for authentication services provided by the X.500 directory.
 - ✓ *The directory acts as a repository of public-key certificates.*
- X.509 also defines **authentication protocols** based on public-key certificates.

X.509:

- First issued in 1988
- Revised in 1993 to address security concerns
- Version 3 issued in 1995 and revised in 2000
- Based on **public-key cryptography and digital signatures**
- Recommends RSA, but does not mandate a specific algorithm
- **Requires a hash function for digital signatures**

Certificates in X.509

- A public-key certificate is associated with each user
- Certificates are created by a trusted CA and stored in the directory by the CA or the user.
- The directory server neither create public keys nor perform certification and only provides access to certificates.
- Once an X.509 certificate is provided to a user by the certified authority, that certificate is attached to it like an identity card.
- The chances of someone stealing it or losing it are less, unlike other unsecured passwords.

Working/use of an X.509 public-key certificate.

PART 1: Creating an X.509

PART 2: Verifying an X.509 Certificate

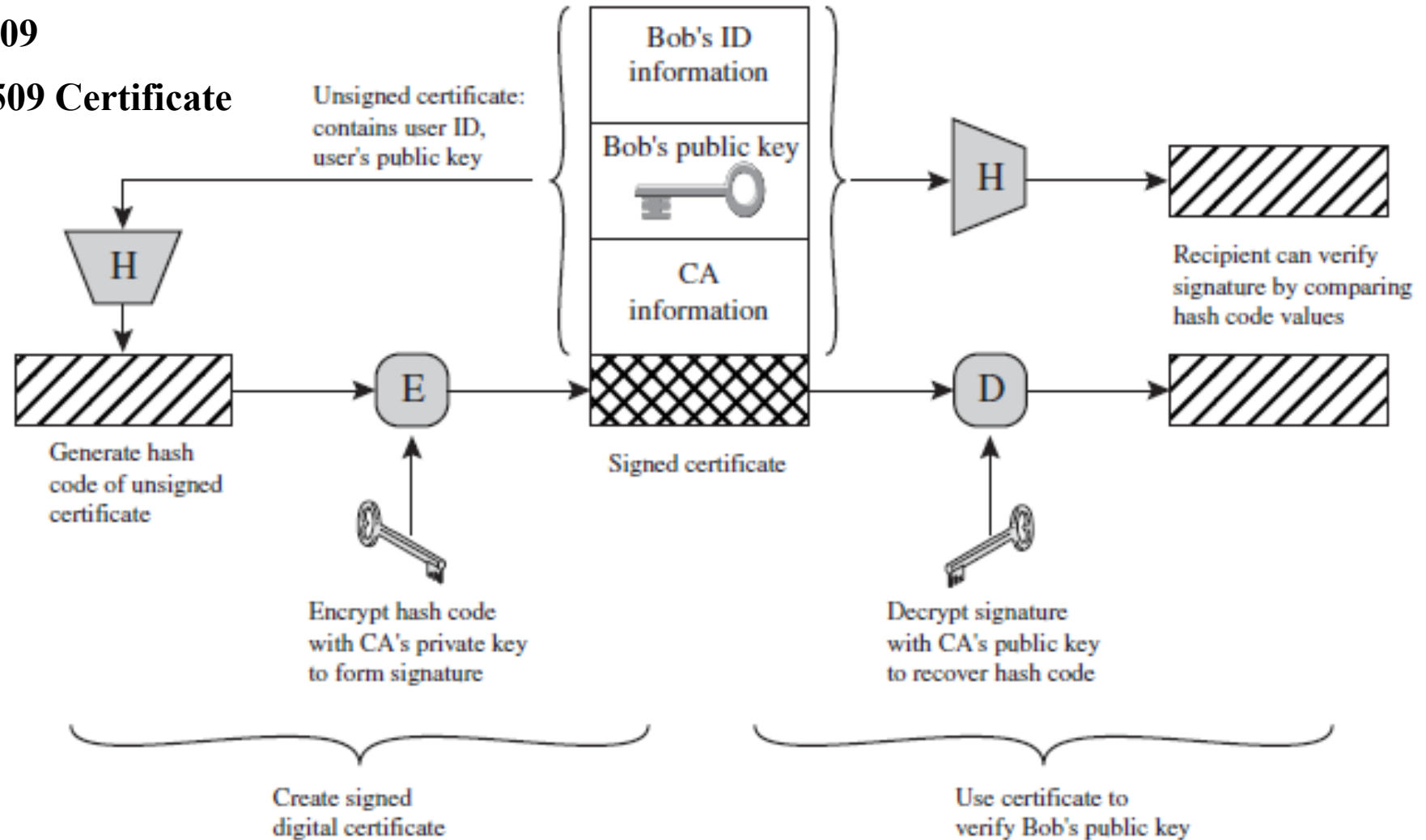


Figure 14.13 Public-Key Certificate Use

PART 1: Creating an X.509 Certificate

Step 1: Unsigned certificate (X.509 data)

- This box contains:
 - ✓ **Bob's ID information** → *Subject Name*
 - ✓ **Bob's public key** → *Subject's public-key information*
 - ✓ **CA information** → *Issuer Name*

These are exactly the **fields inside an X.509 certificate** and at this stage, it is **NOT trusted yet**.

Step 2: Hashing (H)

- The CA applies a **hash function (H)** to the unsigned certificate.
- This produces a **hash value** of all certificate fields.

This corresponds to “Signature covers all of the other fields of the certificate”

Step 4: Signed X.509 certificate

✓ Now the certificate contains:

- Certificate data (Bob's ID, public key, CA name, validity, etc.)
- **CA's digital signature**

✓ This is now a **valid X.509 authentication certificate**.

PART 2: Verifying an X.509 Certificate

Step 5: Decrypt signature (D)

- The recipient uses **CA's public key**.
- Decrypts the signature to get the **original hash**.
- Why this works: Only the CA's **private key** could have created that signature.

Step 6: Hash again (H)

- The recipient hashes the received certificate data again.
- Generates a **new hash**.

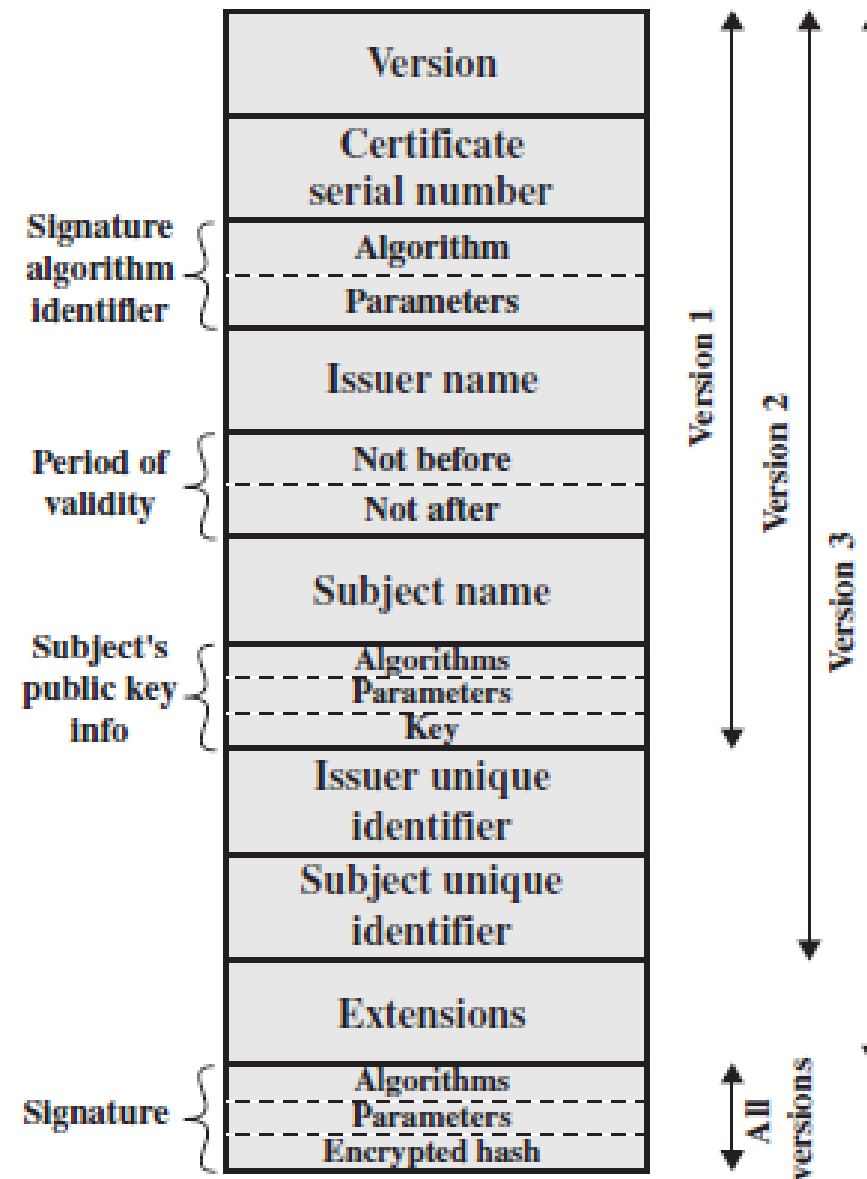
Step 7: Compare hash values

- If both hash values match:
 - Certificate is authentic
 - Bob's public key is trusted
 - Certificate was not altered
- This confirms that the Recipient can verify signature by comparing hash code values.

What X.509 authentication achieves:

- ✓ **Verifies identity of the subject (Bob)**
- ✓ **Verifies authenticity of public key**
- ✓ **Ensures integrity of certificate data**
- ✓ **Ensures trust via CA**

General Format of an X.509 Certificate



(a) X.509 certificate

Figure 14.14 X.509 Formats

1. Version

- ✓ Identifies the version of the certificate format.
- ✓ Default is **Version 1**.
- ✓ **Version 2**: Used if issuer unique identifier or subject unique identifier is present.
- ✓ **Version 3**: Used if one or more extensions are present.

2. Serial Number

- ✓ A unique integer value assigned by the **Certificate Authority (CA)**.
- ✓ Uniquely identifies the certificate issued by that CA.

3. Signature Algorithm Identifier

- ✓ Specifies the algorithm used by the CA to sign the certificate.
- ✓ Includes any associated parameters and this information is repeated in the signature field.

4. Issuer Name

- ✓ The **X.500 name** of the Certificate Authority.
- ✓ Identifies the CA that created and signed the certificate.

5. Period of Validity

- ✓ Defines the time period during which the certificate is valid.
- ✓ Consists of:
 - **Not Before** date
 - **Not After** date

6. Subject Name

- ✓ Identifies the user or entity to whom the certificate is issued.
- ✓ The certificate binds this subject to a public key.

7. Subject's Public Key Information

- ✓ Contains the **public key** of the subject.
- ✓ Includes the identifier of the algorithm used with the key.
- ✓ May include associated parameters.

8. Issuer Unique Identifier (Optional)

- ✓ A bit string used to uniquely identify the issuing CA.
- ✓ Used when an X.500 name is reused for different entities.

9. Subject Unique Identifier (Optional)

- ✓ A bit string used to uniquely identify the subject.
- ✓ Used when an X.500 name is reused.

10. Extensions

- ✓ A set of additional fields.
- ✓ Introduced in **Version 3** certificates.
- ✓ Used to add extra information and functionality.

11. Signature

- ✓ Covers all other fields of the certificate.
- ✓ Contains the hash of the certificate data encrypted with the **CA's private key**.
- ✓ Includes the signature algorithm identifier.